

Compte Rendu – TP4 : Shifumi

Délio Duchesne-Brajon

Groupe 1

[Compte Rendu – TP4 : Shifumi](#)

[A. Analyse du problème](#)

[Règles du jeu](#)

[B. Conception de la solution](#)

[C. Réalisation](#)

[Étapes 1 & 2](#)

[Étapes 3 & 4](#)

[Étape 5](#)

[Étapes 6 & 7](#)

[Étapes 8 & 9](#)

[Partie normale \(mode classique\)](#)

[Fin de partie](#)

[Mode puits activé](#)

[E. Retour d'expérience](#)

[F. Conclusion](#)

A. Analyse du problème

L'objectif de ce TP est de programmer en Java un jeu interactif de **Shifumi (Pierre–Feuille–Ciseaux)** opposant l'utilisateur à l'ordinateur.

Le programme doit :

- Gérer les entrées clavier,
- Utiliser des boucles et conditions,
- Intégrer des fonctions aléatoires,
- Déterminer un gagnant selon des règles définies,

- Offrir une expérience interactive complète avec plusieurs manches et possibilité de rejouer.

Règles du jeu

- Feuille bat Pierre
- Pierre bat Ciseau
- Ciseau bat Feuille
- Choix identiques → Égalité

```

1  /**
2   * La classe principale pour le jeu Shifumi ( pierre - feuille - ciseaux )
3   * ce programme simule un jeu contre l'ordinateur avec des règles classiques
4   * règles du jeu :
5   * feuille bat la pierre
6   * pierre bat les ciseaux
7   * ciseaux bat la feuille
8   * si deux items similaires alors égalité
9   * @author Délia Duchesne-Brajon
10  * @version 1.8
11  */
12
13  public class Shifumi {
14
15      public static void main(String[] args) {
16          // TODO Auto-generated method stub
17      }
18  }
19

```

B. Conception de la solution

La solution est découpée en plusieurs **sous-problèmes logiques** :

1. **Initialisation & choix du nombre de manches** — définir une variable `nbpoints` et valider la saisie (3, 5 ou 10).
2. **Saisie du choix joueur** — demander à l'utilisateur de saisir `P`, `F` ou `C` avec boucle de validation.
3. **Choix aléatoire de l'ordinateur** — générer un nombre aléatoire et le convertir en `P`, `F` ou `C`.
4. **Suspense et affichage** — simuler une réflexion avec `Thread.sleep()` avant d'afficher le choix de l'ordinateur.

5. **Détermination du gagnant** — comparer les deux choix et incrémenter le score du gagnant.
6. **Boucle de partie** — répéter les manches jusqu'à ce qu'un joueur atteigne le score cible.
7. **Rejouabilité** — proposer de recommencer une partie via une boucle sur une variable `reponse`.
8. **Bonus graphiques** — ajouter des icônes pour rendre le jeu plus lisible.
9. **Mode "Puits"** — proposer un mode alternatif avec une nouvelle règle (puits bat pierre et ciseau).

C. Réalisation

L'implémentation a été faite en Java étape par étape.

```

System.out.println(" En combien de points se déroule la partie ? ");

    nbpoints=Scanner.nextInt();
    System.out.println("Vous avez saisi " + nbpoints);
} // fin du while
//étape 2 : choix du joueur
System.out.println("Pierre, Feuille ou Ciseaux ?");
while(choix!='P' && choix!='F' && choix!='C') {
    choix = Scanner.next().charAt(0);
    System.out.println("Vous avez choisi " + choix);

} //fin du while

Scanner Scanner = new Scanner(System.in);
// étape 1 : Afficher et demander à l'utilisateur le nombre de point de la partie en cours
while(nbpoints !=3 && nbpoints !=5 && nbpoints !=10) { // Tant que la saisie n'est pas 3, 5
    System.out.println(" En combien de points se déroule la partie ?"); //Afficher à l'utilisate

    nbpoints=Scanner.nextInt();
    System.out.println("Vous avez saisi " + nbpoints);
} // fin du while
//étape 2 : choix du joueur

```

Initialisation et choix du nombre de manches

→ Montre la déclaration de `nbpoints`, la boucle `while` de validation et la saisie du choix joueur (P, F, C).

```

//fin du while
//étape 3 : Choix de l'ordinateur
aleatoire= (int)(Math.random()*3)+1;
//condition d'attribution de P, F ou C a partir du random
if (aleatoire==1) {
    choixordi='F';
}
else if (aleatoire==2){
    choixordi='P';
}
else{
    choixordi = 'C';
}

```

```

System.out.println("L'ordinateur a choisi" + choixordi);

```

```

//étape 4 : attente de 3 seconde pour le suspense
System.out.println("L'ordinateur à choisi ...");
/ try catch permet de continuer le programme en cas de probleme de fonction thread
try {
    Thread.sleep(3000);
} catch (InterruptedException e) {

    //TODO auto-generated catch block
    e.printStackTrace();
}
System.out.println("Fin du suspense !");
System.out.println("L'ordinateur a choisi" + choixordi);

}

```

Choix aléatoire de l'ordinateur et suspense

→ Montre le code qui génère le nombre aléatoire + le bloc `Thread.sleep(3000)` et l'affichage du choix.

```

72
73
74 //étape 5 : déterminer qui gagne le match
75 // déterminer si il y a une égalité
76 if (choix==choixordi) {
77     System.out.print("égalité");
78 }
79 else if ((choix=='P' && choixordi=='C') ||
80          (choix=='C' && choixordi=='F') ||
81          (choix=='F' && choixordi=='P')) {
82     System.out.println("Vous avez gagné la manche");
83     scoreJoueur++;
84 }
85 else {
86     System.out.println("Vous avez perdu");
87     scoreOrdi++;
88 }
89 System.out.println("Score : Joueur"+scoreJoueur+" - Score ordi :"+scoreOrdi);
90 }
91
92
93
94 // fin du void
95
96 }
97
98 // fin du class
99

```

Problems @ Javadoc Declaration Console X Eclipse IDE for Java Developers 2025-09 Release

<terminated> Shifumi [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\j

```

3
Vous avez saisi 3
Pierre, Feuille ou Ciseaux ?
f
Vous avez choisi f
Feuille
Vous avez choisi F
L'ordinateur à choisi ...
Fin du suspense !
L'ordinateur a choisiC
Vous avez perdu
Score : Joueur0 - Score ordi :1

```

Détermination du gagnant

→ Montre les conditions **if/else** qui comparent le choix joueur et ordinateur, ainsi que l'incrémentation de **scoreJoueur** ou **scoreOrdi**.

```
while (scoreJoueur < nbpoints && scoreOrdi < nbpoints) {
```

```
    // Étape du choix du joueur
```

```
    choix = 'X';
```

```
    while (choix != 'P' && choix != 'F' && choix != 'C') {
```

```
        System.out.println("Faites votre choix : P (Pierre), F (Feuille), C (Ciseau)");
```

```
        choix = scanner.next().toUpperCase().charAt(0);
```

```
        System.out.println("Vous avez choisi : " + symbole(choix));
```

```
    }
```

```

// Étape du choix de l'ordinateur

System.out.println("L'ordinateur réfléchit...");

Thread.sleep(1000);

choixAlea = r.nextInt(3);

if (choixAlea == 0) {

    choixOrdi = 'P';

} else if (choixAlea == 1) {

    choixOrdi = 'F';

} else {

    choixOrdi = 'C';

}

System.out.println("L'ordinateur a choisi : " + symbole(choixOrdi));

// Détermination du gagnant et attribution du point

if (choix == choixOrdi) {

    gagnant = "Égalité";

} else if ((choix == 'P' && choixOrdi == 'C') ||

    (choix == 'C' && choixOrdi == 'F') ||

    (choix == 'F' && choixOrdi == 'P')) {

    gagnant = "Vous avez gagné !";

    scoreJoueur++;

} else {

    gagnant = "L'ordinateur a gagné";

    scoreOrdi++;

}

```

```
// Affichage du résultat et des scores

System.out.println("Résultat : " + gagnant);

System.out.println("Score - Vous : " + scoreJoueur + " | Ordinateur : " + scoreOrdi);

System.out.println("-----");
}
```

```
if (scoreJoueur == nbpoints) {
    System.out.println(" Félicitations, vous avez remporté la partie !");
} else {
    System.out.println(" L'ordinateur a gagné la partie.");
}
```

Et le code qui demande si on veut rejouer une partie (on doit d'abord rajouter un char reponse pour pouvoir répondre oui ou non) :

```
System.out.println("Souhaitez-vous rejouer ? (O/N)");
reponse = scanner.next().toUpperCase().charAt(0);

if (reponse != 'O') {
    reponse = 'N'; // On force la sortie si ce n'est pas 'O'
}
```

Enfin le code qui fait rejouer ou non selon la réponse (il s'agit d'une boucle qui dit que tant que la réponse est O alors ça joue, puis à la fin on laisse une chance au joueur de changer l'état de la réponse pour l'arrêter s'il veut) :

```
char reponse = 'O';

while (reponse == 'O') {
    int nbpoints = 0;
    int scoreJoueur = 0;
    int scoreOrdi = 0;
    char choix = 'X';
    char choixOrdi = 'X';
    int choixalea;
    String gagnant = null;
    char mode = 'X';
    boolean avecPuits = false;
```

Boucle de partie et rejouabilité

→ Montre la boucle `while(scoreJoueur < nbpoints && scoreOrdi < nbpoints)` et le bloc qui propose de rejouer (`reponse == 'O'`).

```

public static String symbole(char choix) {
    if (choix == 'P') {
        return "x (Pierre)";
    } else {
        if (choix == 'F') {
            return "_ (Feuille)";
        } else {
            if (choix == 'C') {
                return "> (Ciseau)";
            } else {
                return "?";
            }
        }
    }
}
}

```

E. Retour d'expérience

Ce TP m'a permis de renforcer :

- L'utilisation de boucles en conditions dans des cas pratiques,
- La validation des entrées utilisateur et la gestion d'erreurs simples,
- La génération aléatoire et l'affichage dynamique,
- L'organisation structurée du code en étapes claires,
- La mise en place d'une boucle de partie et d'une rejouabilité sans bug,
- L'ajout d'options bonus pour enrichir l'expérience utilisateur.

F. Conclusion

Ce TP est un excellent exercice pour appliquer les bases de la programmation Java dans un mini-projet complet.

La progression logique permet de construire une solution robuste et lisible, tout en intégrant des fonctionnalités avancées comme le mode "Puits".

Les captures d'écran et les tests démontrent le bon fonctionnement de chaque étape.

